

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TUẦN 12
MÔN THỰC TẬP CƠ SỞ

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

MỤC LỤC

1. Mục tiêu công việc trong tuần	3
2. Nội dung công việc đã thực hiện	3
2.1 Đo lường và phân tích độ trễ hệ thống AI (Performance Profiling)	3
2.2 Quốc tế hóa ứng dụng (i18n - Bilingual UI)	4
2.3 Tái cấu trúc cơ chế Điều hướng và Quản lý trạng thái toàn cục (Global State Management)	4
3. Kết quả đạt được	4
4. Khó khăn gặp phải	6
5. Kế hoạch tuần tiếp theo	8

1. Mục tiêu công việc trong tuần

Nhiệm vụ trọng tâm tuần này tập trung vào việc đánh giá hiệu năng thực tế của mô hình Deep Learning và nâng cấp trải nghiệm ứng dụng. Cụ thể:

- **Kiểm định hiệu suất AI:** Xây dựng bộ công cụ đo lường để bóc tách thời gian thực thi (Latency) của từng giai đoạn trong quá trình nhận diện (Tiền xử lý, Suy luận, Ghi dữ liệu). Mục đích là để tìm ra "nút thắt cổ chai" (bottleneck) của hệ thống.
- **Thêm ngôn ngữ mới:** Triển khai hệ thống Đa ngôn ngữ (Tiếng Việt - Tiếng Anh) cho toàn bộ ứng dụng từ văn bản tĩnh, nút bấm đến các biểu đồ phân tích dữ liệu, giúp ứng dụng có khả năng tiếp cận tệp người dùng quốc tế.
- **Cân bằng giữa Kỹ thuật và Trải nghiệm:** Đảm bảo việc thêm các tính năng đo lường kỹ thuật không làm ảnh hưởng đến sự tinh gọn, trực quan của giao diện người dùng phổ thông.

2. Nội dung công việc đã thực hiện

2.1 Đo lường và phân tích độ trễ hệ thống AI (Performance Profiling)

Thay vì chỉ hiển thị kết quả phân loại, em đã can thiệp sâu vào luồng thực thi của mô hình EfficientNet tại tệp `page_scanner.py`. Sử dụng thư viện `time` của Python, em đặt các "trạm bấm giờ" để đo lường 3 chặng xử lý cốt lõi:

- **Tiền xử lý Tensor (Preprocessing):** Đo thời gian từ lúc nhận ảnh PIL đến khi chạy qua hàm `transform()` để đưa lên thiết bị (DEVICE).
- **AI Suy luận (Inference):** Đo thời gian khởi lệnh `torch.no_grad()` thực thi việc truyền feed-forward qua mạng nơ-ron và xuất ra các vector xác suất (softmax).
- **Thao tác I/O & CSDL (Database Ops):** Đo thời gian lưu ảnh vật lý vào ổ cứng và gọi hàm `save_to_db` để ghi vào SQLite.

Kết quả thời gian (tính bằng mili-giây - ms) được hiển thị trực tiếp. Tuy nhiên, để tuân thủ nguyên tắc thiết kế UI/UX tinh gọn, em đã bọc toàn bộ bộ đo lường này vào một thẻ đóng/mở (`st.expander`). Điều này giúp giao diện chính không bị rườm rà, nhưng giảng viên/nhà phát triển vẫn có thể kiểm tra thông số kỹ thuật bất cứ lúc nào.

2.2 Quốc tế hóa ứng dụng (i18n - Bilingual UI)

Để chuyển đổi ngôn ngữ linh hoạt mà không bị độ trễ (delay) từ các API dịch máy bên ngoài, em đã áp dụng tiêu chuẩn **i18n**:

- **Xây dựng Dictionary cục bộ:** Toàn bộ khối văn bản hướng dẫn xử lý rác được em biên soạn lại và tách thành hai bộ từ điển tĩnh `ADVICE_VI` và `ADVICE_EN`.
- **Ánh xạ dữ liệu biểu đồ:** Tại trang Thống kê (`page_dashboard.py`), dữ liệu trả về từ SQLite là Tiếng Việt. Để biểu đồ Plotly hiển thị chuẩn Tiếng Anh khi người dùng chuyển đổi, em đã sử dụng hàm `.map()` của thư viện Pandas để ánh xạ (map) các nhãn "Tái Chế", "Hữu Cơ"... sang "Recyclable", "Organic"... ngay trên không gian bộ nhớ RAM (DataFrame) trước khi vẽ biểu đồ.
- **Đồng bộ hóa UI:** Một biến trạng thái `st.session_state['lang']` được thiết lập chạy ngầm toàn hệ thống. Mọi nút bấm, hộp thoại cảnh báo (`st.warning`, `st.success`) đều được kiểm tra qua mệnh đề điều kiện `if lang == 'vi'` để hiển thị chuỗi ký tự (string) tương ứng.

2.3 Tái cấu trúc cơ chế Điều hướng và Quản lý trạng thái toàn cục (Global State Management)

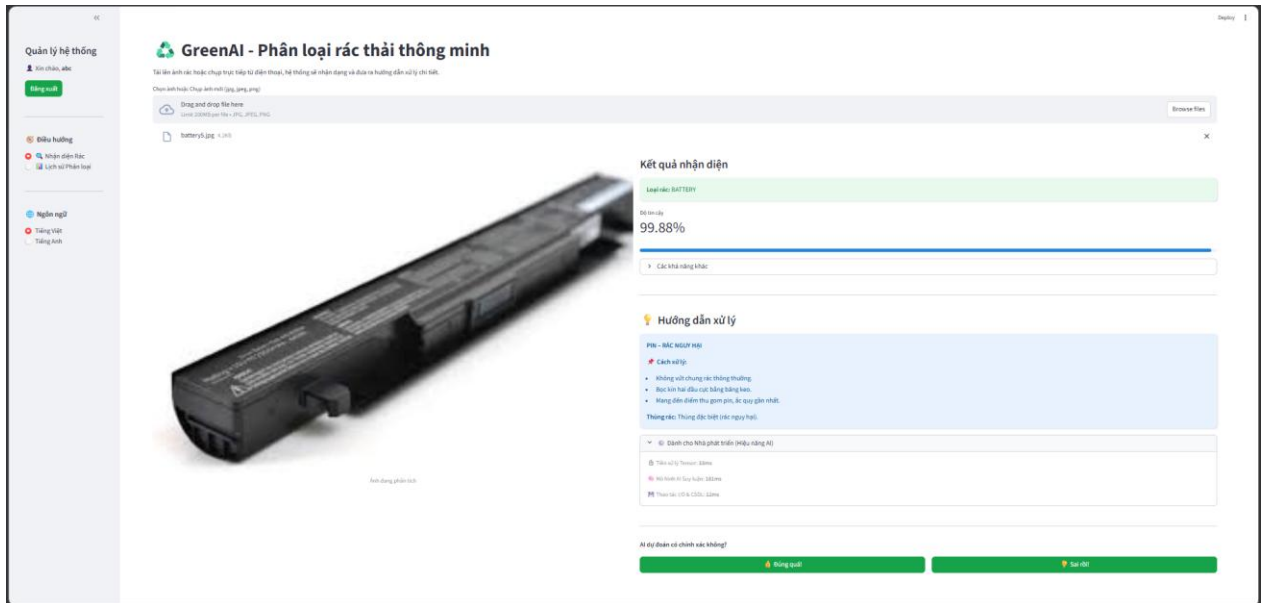
Để thanh Menu bên trái (Sidebar) đồng bộ ngôn ngữ tức thời với các trang chức năng, em đã gom toàn bộ logic vẽ Sidebar từ các tệp con về duy nhất tệp chính `app.py`. Nhờ vậy, Menu điều hướng và Menu đổi ngôn ngữ hiển thị cực kỳ chuyên nghiệp và đồng nhất.

3. Kết quả đạt được

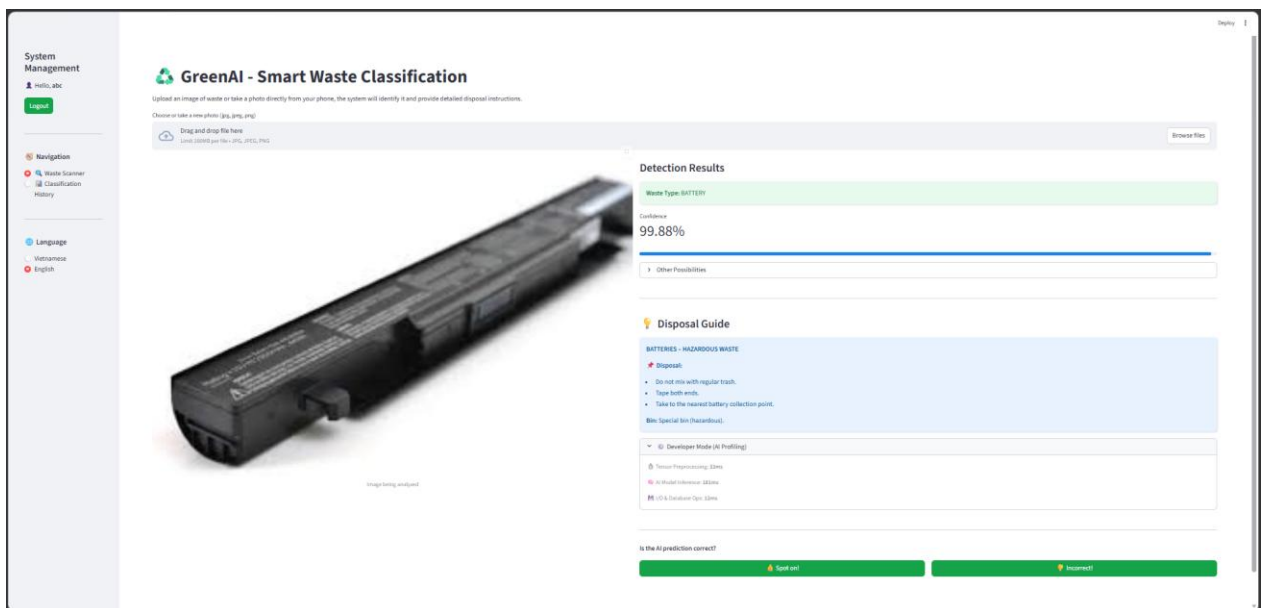
Kết thúc Tuần 12, hệ thống đã có một bước nhảy vọt về mức độ hoàn thiện:

- **Hệ thống Profiling chính xác:** Ứng dụng đã có khả năng tự chẩn đoán tốc độ xử lý. Thực tế kiểm thử cho thấy AI suy luận rất nhanh (nhờ `torch.no_grad()`), trong khi thao tác ghi I/O tốn nhiều thời gian hơn, mở ra hướng tối ưu hóa bộ nhớ ở các tuần sau.
- **Giao diện Song ngữ (Bilingual) mượt mà:** Người dùng có thể chuyển đổi qua lại giữa Tiếng Việt và Tiếng Anh ngay lập tức (zero-latency). Toàn bộ nội dung từ thanh Sidebar, bảng dữ liệu, đến các biểu đồ phân tích đều thay đổi đồng bộ, mang lại trải nghiệm phần mềm chuẩn quốc tế.

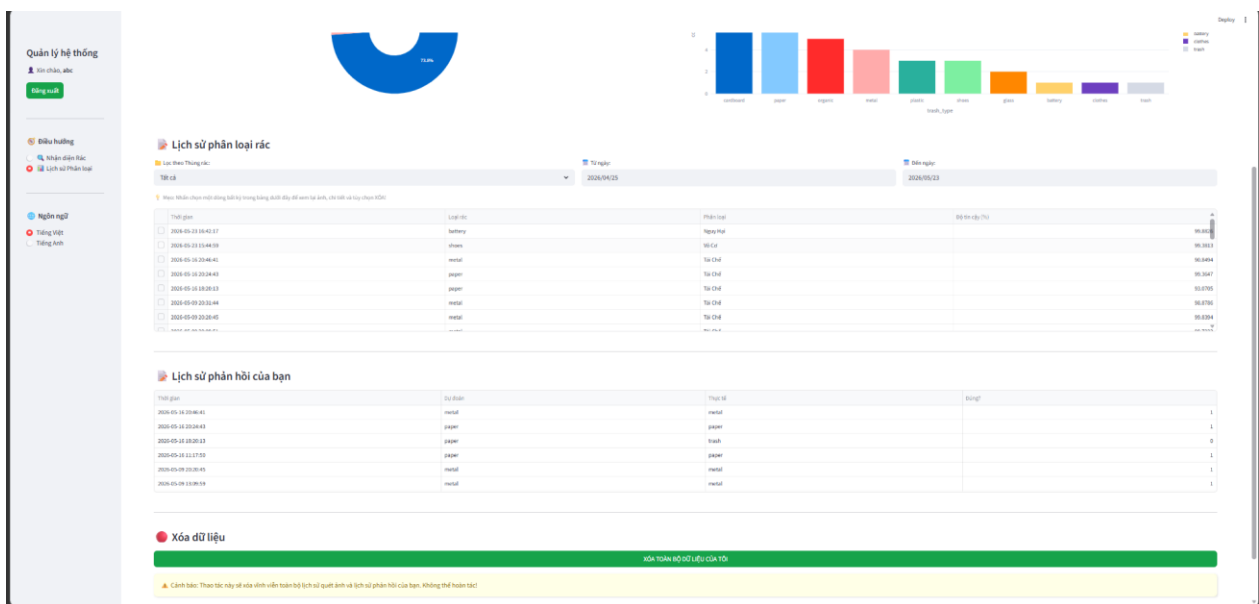
- **Trải nghiệm người dùng (UX) không bị gián đoạn:** Ứng dụng giữ được tính thẩm mỹ cao, phân tách rõ ràng không gian sử dụng của người dùng phổ thông (End-user) và kỹ sư phát triển (Developer).



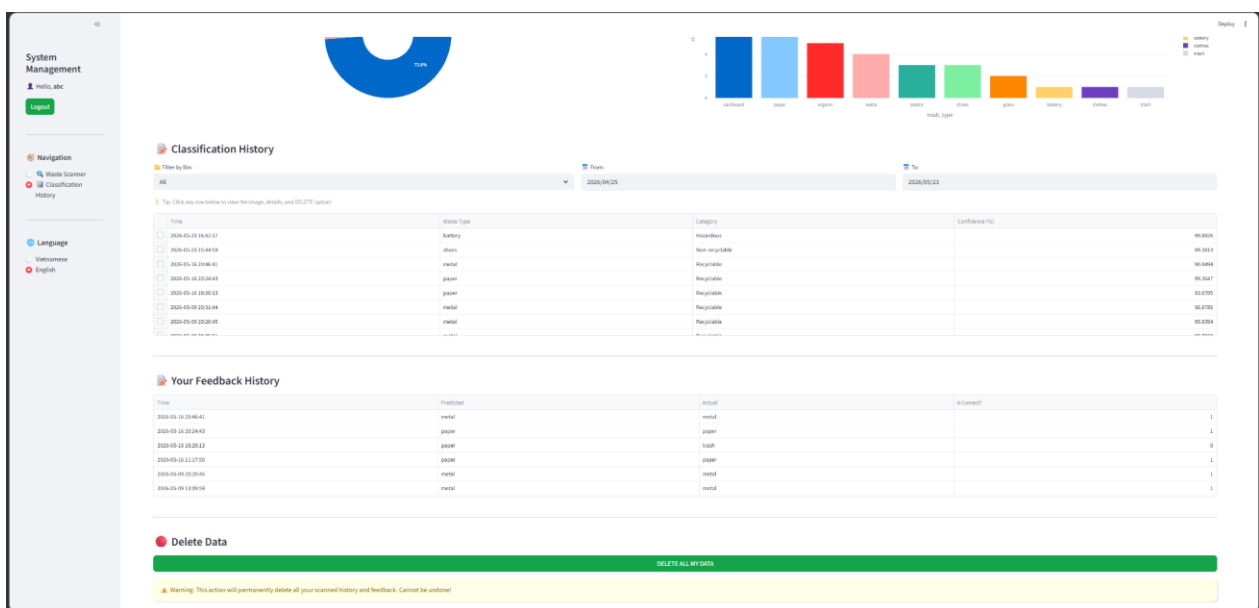
Hình 1 - Giao diện Nhận diện rác (Tiếng việt)



Hình 2 - Giao diện Nhận diện rác (Tiếng anh)



Hình 3 - Giao diện Lịch sử phân loại (Tiếng việt)



Hình 4 - Giao diện Lịch sử phân loại (Tiếng anh)

4. Khó khăn gặp phải

Trong tuần này, khi giải quyết bài toán Đa ngôn ngữ, em đã vấp phải 2 khó khăn kỹ thuật đặc thù của nền tảng Streamlit và đã đúc kết được những bài học quan trọng:

Khó khăn 1: Quyết định giữa API dịch tự động (Auto-Translation API) và Từ điển tĩnh (Static Dictionary)

- **Mô tả vấn đề:** Ban đầu, em cân nhắc việc tích hợp Google Translate API để tự động dịch các nhãn văn bản nhằm tiết kiệm công sức gõ code.
- **Quá trình phân tích và Khắc phục:** Tuy nhiên, khi đánh giá tính khả thi, việc gọi API bên ngoài (Third-party API) sẽ tạo ra độ trễ (latency) từ 1-3 giây mỗi khi người dùng tương tác, làm ứng dụng bị giật (lag). Hơn nữa, máy dịch tự động thường làm hỏng các cú pháp Markdown (như in đậm, xuống dòng) hoặc dịch sai các thuật ngữ chuyên ngành (ví dụ: "Rác vô cơ" bị dịch nhầm thành "Inorganic" thay vì "Non-recyclable").
- **Bài học rút ra:** Đối với kiến trúc giao diện tĩnh (Static UI), việc sử dụng các tệp tài nguyên từ điển (i18n dictionaries) được kiểm soát thủ công là tiêu chuẩn bắt buộc. Nó đảm bảo tốc độ phản hồi tức thời (0 ms) và kiểm soát 100% độ chính xác của ngôn ngữ chuyên ngành.

Khó khăn 2: Lỗi "Mất phương hướng" (Routing Reset) khi thao tác với Menu chữ

- **Mô tả vấn đề:** Em gặp một lỗi logic nghiêm trọng: Bất cứ khi nào đang xem trang "Lịch sử phân loại" mà bấm đổi ngôn ngữ sang Tiếng Anh, ứng dụng lại tự động "văng" ngược về trang "Nhập diện rác".
- **Quá trình phân tích và Khắc phục:** Sau quá trình debug, em phát hiện nguyên nhân đến từ cơ chế lưu trạng thái bằng văn bản của Streamlit. Khi thanh Menu đang hiển thị chữ "Lịch sử Phân loại", việc đổi ngôn ngữ làm nó biến thành "Classification History". Vì chuỗi ký tự bị thay đổi, Streamlit không tìm thấy giá trị cũ, dẫn đến việc tự động reset về giá trị đầu tiên của danh sách. Em đã tái cấu trúc cơ chế điều hướng bằng cách loại bỏ sự phụ thuộc vào chuỗi văn bản bề mặt. Thay vào đó, em tạo ra một biến trạng thái ngầm `st.session_state['current_page']`. Biến này chỉ lưu giá trị logic (ví dụ: 'scanner' hoặc 'dashboard'). Dù giao diện có đổi sang thứ tiếng nào, hệ thống vẫn "nhớ" chính xác vị trí người dùng đang đứng.
- **Bài học rút ra:** Không được dùng giá trị hiển thị (UI Value / Text) để làm biến điều hướng luồng xử lý của hệ thống (Routing Logic). Việc tách biệt rõ ràng giữa Dữ liệu hiển thị và Trạng thái hệ thống (State Management) là chìa khóa để xây dựng các ứng dụng web phức tạp.

5. Kế hoạch tuần tiếp theo

Trong Tuần 13 sắp tới, hệ thống GreenAI sẽ bước vào giai đoạn tối ưu hóa hạ tầng và thiết lập các cơ chế bảo mật cốt lõi trước khi nghiệm thu. Kế hoạch cụ thể bao gồm 2 nhiệm vụ trọng tâm:

- **Cơ chế sao lưu dữ liệu tự động (Database Auto-Backup):** Nghiên cứu và thiết lập luồng chạy ngầm để tự động sao lưu (backup) tệp cơ sở dữ liệu `trash_history.db` sang một thư mục an toàn theo định kỳ. Nhiệm vụ này nhằm đảm bảo khả năng phục hồi hệ thống, phòng ngừa rủi ro mất mát hoặc hỏng hóc dữ liệu do sự cố máy chủ ngoài ý muốn.
- **Kiểm soát và bảo mật file tải lên (Input Validation & Security):** Xây dựng lớp bẫy lỗi (Validation) tại giao diện tải ảnh để tự động kiểm tra định dạng và giới hạn dung lượng tệp tin (ví dụ: chặn các file lớn hơn 5MB). Tính năng này giúp bảo vệ tài nguyên lưu trữ, tiết kiệm băng thông và ngăn chặn các nguy cơ quá tải bộ nhớ đệm (DoS) do người dùng tải lên các tệp rác cỡ lớn.